

Harnessing the Power of Multi-Agent Systems for Generative AI (Part 1)



Moritz Georg Glauner ,



Lion Schulz

| Last modified: 25.03.2024

| 6 minutes read



As generative AI tackles increasingly complex tasks, multi-agent systems provide a promising approach by dividing the workload among specialized agents. In this blog post, we explore the potential of multi-agent systems in generative AI, discuss when they are most effective, and introduce Autogen, the first framework designed for creating multi-agent setups. By understanding the building blocks and implementation of multi-agent systems using Autogen, you will gain insights into leveraging this powerful tool for automating complex processes and enhancing the capabilities of generative AI.

Intro

In 2023, large language models became widely available, and most users focused on **retrieval augmented generation** use cases. In 2024, the question arises about the next use cases once all users have access to

a chat tool and their own document archive can be queried via a chatbot.

The use cases for generative AI are becoming more **specific** and **complex**. These include:

- Integrating databases or *non-textual* information,
- Automating *non-deterministic* processes like publishing job vacancies or reference slides.

While the possibilities for generative AI are *endless*, **practical and stable implementation** presents challenges.

This blog post discusses the **benefits of collaborative work among multiple agents**. It is the *first of two parts*. This part explains the building blocks of a multi-agent system and how to implement them using Autogen, the first library designed for multi-agent setups. The second part will focus on LangGraph, a recently released library for implementing multi-agent workflows based on the popular LangChain library.

When is it necessary to use multiple agents?

It is important to note that multiple agents do *not necessarily improve the quality of work on a single task*. However, if the task can be broken down into **subtasks**, then having several **specialized agents** working together can be an effective approach.

For instance:

- If you aim to *automate customer enquiries* but the model does not match the company's communication style, it is worth investing time in **fine-tuning**.
- If you need to *query and summarize information from multiple sources* and validate it according to the corporate communication policy, it may be useful to **divide the process among several agents**.

ChatGPT and other language models can *generate text* and have been fine-tuned for **function calling**. They understand the concept of function calls but *cannot directly execute them*. Instead, they can only indicate which function to call with specific parameters, requiring a corresponding runtime for execution.

Combining this with the intelligence of the language model creates an agent capable of:

1. **Independently creating** program code
2. **Executing** program code
3. **Debugging** program code based on error messages.

This agent is a *powerful tool* as it can not only generate textual output but also **plan and execute actions independently**. In principle, such agents can be combined with a wide variety of tools, for example:

- To search for information from a database,
- To query a WebAPI
- To analyze images or

- To create a website.

It is possible to combine these diverse tools into a **single agent for execution**, but *compromises must be made*.

- Certain tasks demand a high-quality and costly language model, while others can be accomplished with a smaller model that has been fine-tuned.
- Even if a single model is used for all tasks, different configurations are typically recommended.
 - A *low temperature value* is recommended for programming tasks
 - A *higher temperature value* is recommended for creative writing
- Agents that extract information from a database require **detailed knowledge of table schemas**.

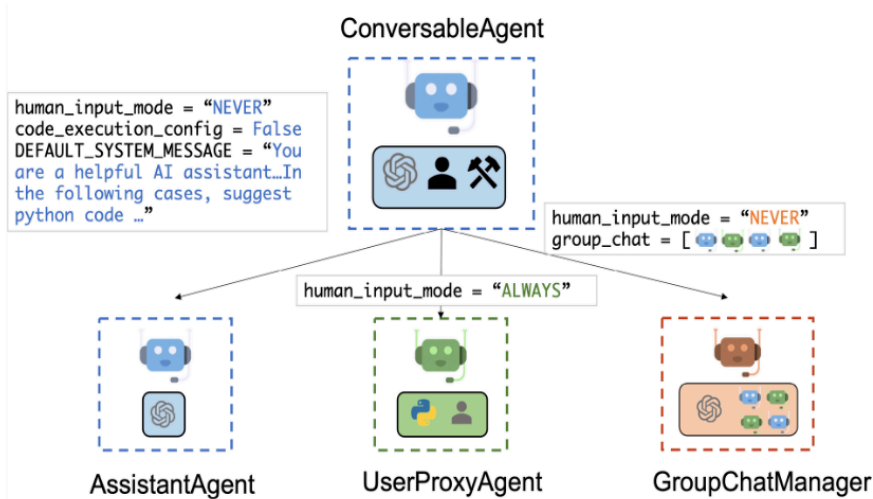
Different tools may compete in this area. It is important to consider why an agent should be able to perform multiple tasks simultaneously when the *UNIX and microservice philosophy* suggests that different tasks should be handled separately.

Autogen - the first framework for multi-agent systems

Autogen is a framework for creating multi-agent systems. It is based on a [paper presented in 2023](#) and is available as an [open-source repository from Microsoft](#).

In the Autogen concept, multi-agent systems are **group conversations** between different conversable agents, which are manifested by a Python class. There are three types of Conversable Agents:

- **UserProxyAgent**, which represents the user. The agents send the user's input into the conversation and return the result. They also execute functions called by other agents.
- The **GroupChatManager** manages the flow of conversations between the agents. The default setting uses a simple *round-robin heuristic*. However, more complex configurations are possible, where a language model determines the next speaker.
- The **AssistantAgent** contains the content agents, which are configured for specific tasks.



The basic concept is simple: one agent sends a message to another, optionally requesting a reply. If multiple agents are in a group chat, the messages are forwarded to all participants, creating a **shared chat history**. This way, agents have a *shared pool of knowledge* and can build on the work of their predecessors.

Autogen provides [numerous examples](#) of this, for instance:

- A programmer can download relevant papers from an online research repository
- A product manager can generate product ideas from them. However, this procedure can cause the chat history to grow quickly, making queries slower and more expensive.

```

llm_config = {"config_list": config_list_gpt4, "cache_seed": 42}
user_proxy = autogen.UserProxyAgent(
    name="User_proxy",
    system_message="A human admin.",
    code_execution_config={
        "last_n_messages": 2,
        "work_dir": "groupchat",
        "use_docker": False,
    },
    human_input_mode="TERMINATE",
)
coder = autogen.AssistantAgent(
    name="Coder",
    llm_config=llm_config,
)
pm = autogen.AssistantAgent(
    name="Product_manager",
    system_message="Creative in software product ideas.",
    llm_config=llm_config,
)
groupchat = autogen.GroupChat(agents=[user_proxy, coder, pm], messages=[], max_round=
  
```

```

manager = autogen.GroupChatManager(groupchat=groupchat, llm_config=llm_config)
user_proxy.initiate_chat(
    manager, message="Find a latest paper about gpt-4 on arxiv and find its potential
)

```

Many applications require *closer monitoring of conversations* rather than open group discussions among all agents. Autogen can facilitate this, as demonstrated by examples such as a [chess game](#) where two players communicate with the game and with each other.

To achieve this, special **AssistantAgent** classes must be created to register additional reply functions using the `register_reply` method. The first parameter is a filter for the type of message sender to use with this reply function.

For `_generate_board_reply`, the reply function handles the *special logic of the chess game* and returns a tuple with True and the selected move. True indicates that the function has generated a **final response** and no further registered response functions should execute.

```

sys_msg = """You are an AI-powered chess board agent.
You translate the user's natural language input into legal UCI moves.
You should only reply with a UCI move string extracted from the user's input."""

class BoardAgent(autogen.AssistantAgent):
    board: chess.Board
    correct_move_messages: Dict[autogen.Agent, List[Dict]]

    def __init__(self, board: chess.Board):
        super().__init__(
            name="BoardAgent",
            system_message=sys_msg,
            llm_config={"temperature": 0.0, "config_list": config_list_gpt4},
            max_consecutive_auto_reply=10,
        )
        self.register_reply(autogen.ConversableAgent, BoardAgent._generate_board_reply)
        self.board = board
        self.correct_move_messages = defaultdict(list)

    def _generate_board_reply(
        self,
        messages: Optional[List[Dict]] = None,
        sender: Optional[autogen.Agent] = None,
        config: Optional[Any] = None,
    ) -> Union[str, Dict, None]:

```

```

message = messages[-1]
# extract a UCI move from player's message
reply = self.generate_reply(
    self.correct_move_messages[sender] + [message], sender, exclude=[BoardAge
)
uci_move = reply if isinstance(reply, str) else str(reply["content"])
try:
    self.board.push_uci(uci_move)
except ValueError as e:
    # invalid move
    return True, f"Error: {e}"
else:
    # valid move
    m = chess.Move.from_uci(uci_move)
    display( # noqa: F821
        chess.svg.board(
            self.board, arrows=[(m.from_square, m.to_square)], fill={m.from_s
        )
    )
    self.correct_move_messages[sender].extend([message, self._message_to_dict
    self.correct_move_messages[sender][-1]["role"] = "assistant"
    return True, uci_move

```

The logic described can create various **communication patterns**. However, it can become *complicated and confusing*, especially with complex setups. This is because the message flow is defined in multiple classes and depends on the sender and response registration position.

If you need to regulate who receives specific information, you must do so separately within the GroupChatManager. This further complicates *complex communication flows*.

Conclusion

Multi-agent systems are useful for building complex *(semi-)autonomous systems*. They allow for the definition of a **specific agent** for each subtask, including:

- Prompting
- Model selection
- Configuration

Autogen was the first framework to support developers in defining such systems and offers numerous examples, particularly for *open group conversations*. However, if a consciously controlled flow of information is required, Autogen may not be the best choice due to its design.

To improve your understanding, we suggest reading the **second part** of this blog post series, which covers controlling multi-agent LLM systems with *LangGraph*.

The opinions and information stated in this article are personal to the individual author and do not necessarily represent Bertelsmann.

About the Author



Moritz Georg Glauner

Head of Data Science
Corporate Center



Lion Schulz

Head of Machine Learning,
Bertelsmann AI Hub
Corporate Center

Tags

#python #sql #architecture #linux #cloud #collaboration #api #automation #platform #open-source
#visualization #nlp #artificial-intelligence #communication #bcp #user-experience #machine-learning
#sharing #innovation #azure #amazon-web-services #business-application #google-cloud-platform
#kubernetes #data-science #training #api-economy #leadership #data #creativity #bertelsmann
#chatgpt #generative-ai

Share Article

Newest job offers

Senior Digital Project Manager

London, LND, GB, SW1V 2SA

Project Management

AI Engineer

Morrisville, NC, US, 27560

Artificial Intelligence / Machine Learning

Senior Product Manager, AdFormats & Propositions (m/f/d)

Amsterdam, NH, NL, 1021 JI

Product Management

Software Engineering Lead

Stockholm, AB, SE, 11122

Software Development / DevOps

Head of AdFormats & Propositions (m/f/d)

Hamburg, HH, DE, 20457

Product Management

Domain Architect

Hilversum, NH, NL, 1212 AA

System Architecture

Senior Software Engineer

Tallinn, 37, EE, 10412

Software Development / DevOps

Cloud Engineer (GCP)

Morrisville, NC, US, 27560

Project Management

Create Your Own Career



CREATE
YOUR
OWN
CAREER

On our career website "Create Your Own Career" you can discover the wide range of entry and career opportunities at Bertelsmann and be inspired by our employee stories!

Find more interesting jobs [↗](#)

Bertelsmann
Tech Blog

Tech Blog powered by



Bertelsmann
Collaboration Platform

© 2025 Bertelsmann Tech Blog. All rights reserved.

[Imprint](#)

[Privacy Policy](#)

[Privacy Settings](#)

[RSS](#)